

SECURITY PRINCIPLES

[Kaynak](#)

İÇERİK

1. CIA Üçlüsü ve Ötesi
2. DAD Üçlüsü: CIA'in Karşıt Kuvvetleri
3. Güvenlik Modellerinin Temel Kavramları
4. Derinlemesine Savunma
5. ISO/IEC 19249: Mimari ve Tasarım Prensipleri
6. Güven Modelleri: Sıfır Güven ve Güven Ama Doğrula
7. Tehdit ve Risk Kavramları

1. CIA ÜÇLÜSÜ VE ÖTESİ



1. Güvenliğin Temel Taşı: CIA Üçlüsü (The CIA Triad)

Bir sistemin "güvenli" olarak tanımlanabilmesi için güvenlik bileşenlerini derinlemesine anlamamız gerekir. Bir sistemin güvenliğini değerlendirirken referans noktamız **CIA Üçlüsü** (Confidentiality, Integrity, Availability) olmalıdır.

- **Gizlilik (Confidentiality):** Verilere yalnızca yetkili kişilerin veya hedeflenen alıcıların erişebilmesini sağlar. Yetkisiz erişimin engellenmesidir.

- **Bütünlük (Integrity):** Verinin değiştirilememesini ve eğer bir değişiklik yapılmışsa bunun tespit edilebilmesini garanti eder. Verinin doğruluğunun ve tamlığının korunmasıdır.
- **Erişilebilirlik (Availability):** Sistemin veya hizmetin ihtiyaç duyulduğu anda erişilebilir ve kullanılabilir olmasını hedefler.

2. CIA Üçlüsünün Uygulamalı Senaryoları

Soyut tanımları somutlaştırmak için bu üç kavramın **Online Alışveriş** ve **Sağlık Sistemleri** üzerindeki etkilerini inceleyelim.

Senaryo A: Online Alışveriş (E-Ticaret)

Bir e-ticaret işlemi sırasında CIA prensipleri şu şekilde çalışır ve ihlal edilirse şu sonuçlar doğar:

- **Gizlilik (Confidentiality):**
 - Online alışveriş yaparken, kredi kartı numaranızın yalnızca ödemeyi işleyen kurumla paylaşılmasını beklersiniz.
 - **Risk/Sonuç:** Eğer kredi kartı bilgilerinizin güvenilmeyen tarafların eline geçeceğinden şüphelenirseniz, işlemi yapmaktan vazgeçersiniz. Ayrıca, kişisel bilgilerin (kredi kartları dahil) ifşa olduğu bir veri ihlali yaşanırsa, şirket hem itibar hem de finansal açıdan büyük kayıplara uğrar.
- **Bütünlük (Integrity):**
 - Siparişi verdikten sonra, gönderdiğiniz verinin (örneğin teslimat adresi) yolda değişmemesi gerekir.
 - **Risk/Sonuç:** Eğer bir saldırgan gönderdiğiniz teslimat adresini değiştirebilirse, paketiniz başka birine gider. Veri bütünlüğünün olmadığı bir ortamda, satıcıya güvenemez ve sipariş vermekten çekinirsiniz.
- **Erişilebilirlik (Availability):**
 - Sipariş vermek için mağazanın web sitesini veya mobil uygulamasını kullanırsınız.
 - **Risk/Sonuç:** Hizmet erişilebilir değilse ürünlere göz atamaz veya sipariş veremezsiniz. Bu teknik sorunlar devam ederse, kullanıcı olarak pes eder ve rakip bir mağazaya yönelirsiniz.

Senaryo B: Hasta Kayıtları ve Tıbbi Sistemler

Sağlık sektöründe güvenlik ihlalleri finansal kayıptan öte, hayati riskler taşır:

- **Gizlilik (Confidentiality):**
 - Modern ülkelerdeki yasalar gereği, sağlık hizmeti sağlayıcıları tıbbi kayıtların gizliliğini sağlamak zorundadır.
 - **Risk/Sonuç:** Sağlık sağlayıcıları, hastaların tıbbi kayıtlarını yasa dışı bir şekilde ifşa ederse yasal olarak sorumlu tutulurlar ve ağır cezalarla karşılaşabilirler.
- **Bütünlük (Integrity):**

- Hasta verilerinin doğruluğu hayati önem taşır.
- **Risk/Sonuç:** Bir hasta kaydı yanlışlıkla veya kötü niyetle değiştirilirse, hastaya **yanlış tedavi** uygulanabilir. Bu durum hayati tehlike yaratan durumlara yol açabilir. Tıbbi kayıtların bütünlüğü sağlanamazsa sistem sadece işe yaramaz olmakla kalmaz, aynı zamanda tehlikeli hale gelir.
- **Erişilebilirlik (Availability):**
 - Bir hasta kontrol muayenesi için kliniğe geldiğinde sistemin çalışır durumda olması şarttır.
 - **Risk/Sonuç:** Sistemin erişilemez olması, doktorun hastanın geçmiş kayıtlarına ulaşamaması demektir. Doktor, hastanın mevcut semptomlarının geçmiş tıbbi öyküsüyle ilgili olup olmadığını bilemez. Bu durum teşhisi zorlaştırır ve hata yapma olasılığını artırır.

Önemli Not: Güvenlik Dengesi

Her senaryoda üç işleve (CIA) aynı ağırlığı vermek zorunda değiliz.

- **Örnek:** Bir üniversite duyurusu genellikle "gizli" değildir (herkes okuyabilir), ancak belgenin **bütünlüğü** kritiktir (duyurunun içeriği değiştirilmemelidir).

3. CIA'in Ötesi: Orijinallik ve İnkâr Edilemezlik

CIA üçlüsünün ötesine geçildiğinde iki kritik kavram daha karşımıza çıkar:

1. **Orijinallik (Authenticity):** Sahte veya taklit olmama durumudur. Belgenin, dosyanın veya verinin iddia edilen kaynaktan geldiğinin doğrulanmasını sağlar.
2. **İnkâr Edilemezlik (Nonrepudiation):** "Repudiate", bir şeyin geçerliliğini reddetmek anlamına gelir. İnkâr edilemezlik, orijinal kaynağın, söz konusu belgeyi, dosyayı veya veriyi üreten/gönderen kişi olduğunu reddedememesini garanti altına alır.

Neden Bu Kavramlara İhtiyacımız Var? (Ticari Senaryo)

Gerçek dosyaları/siparişleri sahtelerinden ayırt etmek ve karşı tarafın işlemi reddetmesini engellemek ticari sistemlerin kullanılabilirliği için hayati önem taşır.

- **Küçük Ölçekli Örnek (Tişört):** Bir tişört siparişinde, kapıda ödeme seçeneğiyle ürün gönderip, alıcının aslında böyle bir sipariş vermediğini öğrenmeyi bir işletme olarak tolere edebilirsiniz.
- **Büyük Ölçekli Örnek (1000 Araba):** Hiçbir şirket, 1000 arabalık bir sevkiyatı yaptıktan sonra siparişin "sahte" olduğunu öğrenmeyi tolere edemez.
 - **Orijinallik İhtiyacı:** Söz konusu müşterinin bu siparişi gerçekten verip vermediğini doğrulamak istersiniz.
 - **İnkâr Edilemezlik İhtiyacı:** Müşterinin, siparişi verdikten sonra "Ben bunu sipariş etmedim" diyerek durumu inkâr edememesini garanti altına almak istersiniz.

Sonuç: Orijinallik ve inkar edilemezlik olmadan, büyük ölçekli iş süreçleri yürütülemez.

4. Parkerian Hexad (Parker Altılısı)

1998 yılında Donn Parker, CIA üçlüsünü genişleterek altı güvenlik elementinden oluşan bir set önermiştir. Buna **Parkerian Hexad** denir.

Bileşenler:

1. **Erişilebilirlik (Availability)** - *(Daha önce işlendi)*
2. **Bütünlük (Integrity)** - *(Daha önce işlendi)*
3. **Gizlilik (Confidentiality)** - *(Daha önce işlendi)*
4. **Orijinallik (Authenticity)** - *(Daha önce işlendi)*
5. **Fayda / Kullanışlılık (Utility)** - *(Yeni Kavram)*
6. **Sahiplik / Zilyetlik (Possession)** - *(Yeni Kavram)*

Bu modeldeki yeni iki kavramı detaylandıralım:

A. Fayda / Kullanışlılık (Utility)

Bu kavram, bilginin ne kadar yararlı olduğuna odaklanır. Bilgiye erişebiliyor olmanız, onun her zaman işinize yarayacağı anlamına gelmez.

- **Örnek (Şifreli Laptop):** Bir kullanıcı, şifreli depolama alanına sahip bir dizüstü bilgisayarın şifre çözme anahtarını (decryption key) kaybetmiş olsun.
 - Kullanıcının elinde bilgisayar ve disk fiziksel olarak mevcuttur (**Erişilebilirlik** var).
 - Ancak şifreyi çözemediği için verilere anlamlı bir şekilde ulaşamaz.
 - **Sonuç:** Bilgi teknik olarak orada olsa bile, "kullanışlı" bir formda değildir; yani **Utility (Fayda)** kaybedilmiştir.

B. Sahiplik / Zilyetlik (Possession)

Bu güvenlik elementi, bilgiyi yetkisiz alımlara, kopyalamalara veya kontrole karşı korumayı gerektirir. Sahiplik kaybı, verinin kontrolünün başkasına geçmesi demektir.

- **Örnek 1 (Fiziksel Hırsızlık):** Bir saldırgan yedekleme diskinizi (backup drive) çalarsa, disk onlarda olduğu sürece bilginin **sahipliğini (possession)** kaybetmiş olursunuz.
- **Örnek 2 (Fidye Yazılımı - Ransomware):** Bir saldırgan verilerinizi fidye yazılımı ile şifrelemeyi başarırorsa, veriler sizin sisteminizde dursa bile kontrolü saldırgandadır. Bu durum da verinin **sahipliğinin** kaybı anlamına gelir.

2. DAD ÜÇLÜSÜ: CIA'İN KARŞIT KUVVETLERİ

Bir sisteme yönelik güvenlik saldırıları; verinin ifşası, değiştirilmesi veya yok edilmesi yoluyla gerçekleşir. Güvenlik dünyasında "savunma" tarafını temsil eden **CIA Üçlüsü**'nün tam karşısında, "saldırı" veya "başarısızlık" tarafını temsil eden **DAD Üçlüsü** (Disclosure,



1. DAD Üçlüsünün Bileşenleri ve CIA ile İlişkisi

DAD üçlüsünün her bir elemanı, CIA üçlüsünün bir elemanının zıttıdır. Bir güvenlik ihlali bu üç kategoriden biriyle açıklanabilir:

- **İfşa (Disclosure) - Gizliliğin (Confidentiality) Zıttı:**
 - Gizli verilerin yetkisiz kişilerce görülmesi veya açığa çıkmasıdır.
 - Gizli verinin ifşası, doğrudan sisteme yapılmış bir **gizlilik saldırısı** anlamına gelir.
- **Değiştirme (Alteration) - Bütünlüğün (Integrity) Zıttı:**
 - Verinin yetkisiz bir şekilde modifiye edilmesidir.
 - Örneğin, finansal işlemlerde kullanılan bir **çekin bütünlüğü** vazgeçilmezdir. Çek üzerindeki verinin değiştirilmesi (tahrifat), bütünlük ilkesine yapılan bir saldırıdır.
- **İmha / Erişim Engeli (Destruction/Denial) - Erişilebilirliğin (Availability) Zıttı:**
 - Verinin yok edilmesi veya hizmetin erişilemez hale getirilmesidir.
 - Bu durum, sistemin ihtiyaç duyulduğunda yanıt verememesi (denial) anlamına gelir.

Özetle; DAD Üçlüsü (İfşa, Değiştirme, İmha), CIA Üçlüsü'nün (Gizlilik, Bütünlük, Erişilebilirlik) tam tersi yani "negatif" yansımasıdır.

2. DAD Üçlüsünün Sağlık Sistemleri Üzerinden Analizi

Hasta kayıtları ve ilgili sistemler üzerinden DAD saldırılarının nasıl gerçekleştiğini ve yarattığı sonuçları (neden-sonuç ilişkisi içinde) inceleyelim:

A. İfşa (Disclosure)

- **Senaryo:** Çoğu modern ülkede, sağlık hizmeti sağlayıcıları tıbbi kayıtların gizliliğini korumakla yasal olarak yükümlüdür. Bir saldırganın bu tıbbi kayıtların bir kısmını çaldığını ve herkesin görebileceği şekilde internete sızdirdiğini (dumping) varsayalım.
- **Sonuç:** Bu veri ifşa saldırısı sonucunda, sağlık hizmeti sağlayıcısı hem yasal yükümlülüklerin ihlali hem de itibar kaybı nedeniyle büyük zarara uğrayacaktır.

B. Değiştirme (Alteration)

- **Senaryo:** Bir saldırganın hasta tıbbi kayıtlarına erişip bunları değiştirmeyi başardığını düşünelim. Bu, durumun ciddiyeti (gravity) açısından en kritik senaryolardan biridir.
- **Sonuç:** Bu değiştirme saldırısı, hastaya **yanlış tedavi** uygulanmasına yol açabilir. Dolayısıyla, sadece veri bozulmakla kalmaz, sonuç **hayati tehlike** yaratan bir boyuta ulaşır.

C. İmha / Erişim Engeli (Destruction/Denial)

- **Senaryo:** Tamamen kağıtsız (paperless) sisteme geçmiş, yani tüm verilerini dijital ortamda tutan bir tıbbi tesisi ele alalım. Bir saldırgan veritabanı sistemlerini erişilemez hale getirirse ne olur?
- **Sonuç:**
 - Tesis düzgün işleyemez hale gelir.
 - Geçici olarak kağıt kaleme dönseler bile, hastaların geçmiş kayıtlarına ulaşamazlar.
 - Bu erişim engeli saldırısı (denial attack), tüm tesisin işleyişini durdurur (stall).

3. Güvenlik Dengesi (The Security Balance)

İfşa, değiştirme ve imhaya karşı korunmak; aslında gizliliği, bütünlüğü ve erişilebilirliği sürdürmekle eşdeğerdir. Ancak bu koruma sağlanırken hassas bir **denge** kurulmalıdır:

- **Aşırı Güvenlik Riski:** Gizliliği ve bütünlüğü "aşırı" düzeyde korumaya çalışmak, sistemin **erişilebilirliğini** kısıtlayabilir (Sistem o kadar güvenli olur ki, kimse kullanamaz).
- **Aşırı Erişilebilirlik Riski:** Erişilebilirliği "aşırı" artırmak (her yerden, herkesin, her an kolayca girmesi), **gizlilik ve bütünlüğün** kaybedilmesine yol açabilir.

Kritik Ders: İyi bir güvenlik prensibi uygulaması, bu üç unsur (CIA) arasında doğru dengeyi bulmayı gerektirir.

3. GÜVENLİK MODELLERİNİN TEMEL KAVRAMLARI

Daha önce güvenliğin temel yapı taşının **CIA Üçlüsü** (Gizlilik, Bütünlük ve Erişilebilirlik) olduğunu öğrendik. Peki, bir veya daha fazla güvenlik işlevini garanti eden bir sistemi nasıl tasarlarız? Cevap **Güvenlik Modelleri** kullanmaktır.

Bu bölümde üç temel güvenlik modelini inceleyeceğiz:

1. Bell-LaPadula Modeli

2. Biba Bütünlük Modeli
3. Clark-Wilson Modeli

1. Bell-LaPadula Modeli

Temel Hedef: Bu model öncelikli olarak **Gizliliği (Confidentiality)** sağlamayı amaçlar.

Bell-LaPadula, gizliliği korumak için üç ana kural (özellik) belirler:

A. Kurallar (Properties)

- Basit Güvenlik Özelliği (Simple Security Property):**
 - Bu kural **"Yukarı Okuma Yok" (No Read Up)** olarak adlandırılır.
 - Mantığı:** Daha düşük güvenlik seviyesindeki bir özne (kullanıcı/sistem), daha yüksek güvenlik seviyesindeki bir nesneyi (dosya/veri) okuyamaz.
 - Neden:** Bu kural, hassas bilgilere yetkisiz seviyelerden erişilmesini engeller.
- Yıldız (*) Güvenlik Özelliği (Star Security Property):**
 - Bu kural **"Aşağı Yazma Yok" (No Write Down)** olarak adlandırılır.
 - Mantığı:** Daha yüksek güvenlik seviyesindeki bir özne, daha düşük güvenlik seviyesindeki bir nesneye veri yazamaz.
 - Neden:** Bu kural, hassas (yüksek seviyeli) bilgilerin yanlışlıkla veya kasten daha düşük güvenlik seviyesindeki bir alana ifşa edilmesini (sızdırılmasını) önler.
- İsteğe Bağlı Güvenlik Özelliği (Discretionary-Security Property):**
 - Bu özellik, okuma ve yazma işlemlerine izin vermek için bir **Erişim Matrisi (Access Matrix)** kullanır. İlk iki kural ile birlikte çalışır.

Örnek Erişim Matrisi:

Öznelere (Subjects)	Nesne A (Object A)	Nesne B (Object B)
Özne 1	Yazma (Write)	Erişim Yok
Özne 2	Okuma/Yazma	Okuma (Read)

B. Özet ve Çalışma Mantığı

Bell-LaPadula modelinin ilk iki kuralı şu sloganla özetlenebilir: **"Yukarı Yaz, Aşağı Oku" (Write up, Read down).**

- Yukarı Yaz:** Gizli bilgileri daha yüksek güvenlik iznine sahip kişilerle paylaşabilirsiniz.
- Aşağı Oku:** Daha düşük güvenlik iznine sahip kişilerden gelen bilgileri alıp okuyabilirsiniz.

C. Sınırlamalar

Bell-LaPadula modelinin belirli sınırları vardır. Örneğin, bu model modern anlamda **dosya paylaşımını (file-sharing)** yönetmek için tasarlanmamıştır.

2. Biba Bütünlük Modeli (The Biba Integrity Model)

Temel Hedef: Bu model öncelikli olarak **Bütünlüğü (Integrity)** sağlamayı amaçlar.

Biba modeli, bütünlüğü korumak için iki ana kural belirler:

A. Kurallar (Properties)

1. Basit Bütünlük Özelliği (Simple Integrity Property):

- Bu kural "**Aşağı Okuma Yok**" (**No Read Down**) olarak adlandırılır.
- Mantığı:** Yüksek bütünlük seviyesine sahip bir özne, daha düşük bütünlük seviyesine sahip bir nesneden okuma yapmamalıdır. (Güvenilir verinin, güvenilirmez kaynaktan gelen veriyle kirlenmesi önlenir).

2. Yıldız (*) Bütünlük Özelliği (Star Integrity Property):

- Bu kural "**Yukarı Yazma Yok**" (**No Write Up**) olarak adlandırılır.
- Mantığı:** Düşük bütünlük seviyesine sahip bir özne, yüksek bütünlük seviyesine sahip bir nesneye yazma yapmamalıdır. (Güvenilir verinin, güvenilirmez bir özne tarafından değiştirilmesi/bozulması önlenir).

B. Özet ve Karşılaştırma

Bu iki özellik şu sloganla özetlenebilir: "**Yukarı Oku, Aşağı Yaz**" (**Read up, Write down**).

- Bu kural, Bell-LaPadula Modeli'nin **tam tersidir**.
- Bu zıtlık şaşırtıcı değildir; çünkü biri gizliliği (veriyi saklamayı), diğeri ise bütünlüğü (verinin kalitesini/doğruluğunu) korumaya odaklanır.

C. Sınırlamalar

Biba modeli çeşitli sınırlamalara sahiptir. En bilinen örneği, **iç tehditleri (insider threat)** yönetmekte yetersiz kalmasıdır.

3. Clark-Wilson Modeli

Temel Hedef: Bu model de Biba gibi **Bütünlüğü (Integrity)** sağlamayı amaçlar ancak farklı kavramlar kullanır.

Clark-Wilson modeli aşağıdaki dört temel kavram üzerine kuruludur:

1. Kısıtlanmış Veri Ögesi (Constrained Data Item - CDI):

- Bütünlüğünü korumak istediğimiz veri türüdür. (Örn: Veritabanındaki kritik kayıtlar).

2. Kısıtlanmamış Veri Ögesi (Unconstrained Data Item - UDI):

- CDI dışında kalan tüm veri türleridir. Kullanıcı girdileri (user input) veya sistem girdileri buna örnektir.

3. Dönüşüm Prosedürleri (Transformation Procedures - TPs):

- Okuma ve yazma gibi programlanmış işlemlerdir.
- Bu prosedürler, CDI'ların (Kısıtlanmış Veri Öğelerinin) bütünlüğünü koruyacak şekilde tasarlanmalıdır.

4. Bütünlük Doğrulama Prosedürleri (Integrity Verification Procedures - IVPs):

- CDI'ların geçerliliğini ve bütünlüğünü kontrol eden ve garanti altına alan prosedürlerdir.

4. Diğer Güvenlik Modelleri (Ek Okuma)

Sadece üç temel modeli inceledik ancak literatürde keşfedilebilecek birçok ek güvenlik modeli mevcuttur. Bunlardan bazıları şunlardır:

- Brewer and Nash Modeli
- Goguen-Meseguer Modeli
- Sutherland Modeli
- Graham-Denning Modeli
- Harrison-Ruzzo-Ullman Modeli

4. DERİNLEMESİNE SAVUNMA (DEFENCE-IN-DEPTH)

1. Tanım ve Temel Kavram

Derinlemesine Savunma (Defence-in-Depth), tek bir güvenlik önlemine güvenmek yerine, birden fazla seviyeden oluşan bir güvenlik sistemi oluşturmayı ifade eder. Bu yapısından dolayı **Çok Katmanlı Güvenlik (Multi-Level Security)** olarak da adlandırılır.

Bu yaklaşımın temel felsefesi şudur: Bir savunma katmanı başarısız olsa bile, diğer katmanlar tehdidi durdurmalı veya yavaşlatmalıdır.

2. Analoji ile Anlatım: Kilitli Çekmece Örneği

Bu kavramı somutlaştırmak için günlük hayattan bir senaryo üzerinden gidelim:

Önemli belgelerinizi ve pahalı eşyalarınızı sakladığınız **kilitli bir çekmeceniz** olduğunu varsayalım.

A. Tek Katmanlı Yaklaşım (Riskli Durum)

Çekmece kilitlidir. Ancak, bir hırsız ile değerli eşyalarınız arasında duran **tek engel** bu çekmece kilidi olmasını ister misiniz? Eğer hırsız bu kilidi aşarsa, varlıklarınız tamamen korumasız kalır.

B. Çok Katmanlı Güvenlik Yaklaşımı (İdeal Durum)

Derinlemesine savunma mantığıyla düşündüğümüzde, güvenliği katmanlara ayırmayı tercih ederiz. İdeal senaryoda süreç şöyle işler:

1. **Çekmece** kilitli olmalıdır (En iç katman).
2. Çekmecenin bulunduğu **oda kapısı** kilitli olmalıdır.
3. Dairenin **ana giriş kapısı** kilitli olmalıdır.
4. Binanın **dış kapısı** kilitli olmalıdır.
5. Yol boyunca caydırıcılık ve tespit için **güvenlik kameraları** bulunmalıdır (En dış/izleme katmanı).

3. Neden Bu Yaklaşım İhtiyacımız Var?

Bu çoklu güvenlik seviyeleri her hırsız durduramayabilir; ancak sistemin başarısı şu iki noktada yatar:

- **Bloklama:** Hırsızların büyük çoğunluğunu daha ilk katmanlarda engeller.
- **Yavaşlatma:** Engelleri aşmayı başaran profesyonel saldırganları ise ciddi şekilde yavaşlatır. Bu zaman kaybı, saldırının tespit edilmesi ve müdahale edilmesi için savunmacıya fırsat tanır.

5. ISO/IEC 19249: MİMARİ VE TASARIM PRENSİPLERİ

ISO/IEC 19249 standardı, güvenli sistemler oluşturmak için prensipleri iki ana kategoride ele alır: **Mimari Prensip**ler ve **Tasarım Prensip**leri.

A. Mimari Prensip (Architectural Principles)

Bu kategori, sistemin yapısını ve bileşenlerin birbirleriyle nasıl ilişkilendirileceğini belirleyen 5 temel prensibi içerir.

1. Alan Ayrımı (Domain Separation)

İlişkili bileşenlerin (uygulamalar, veriler veya diğer kaynaklar) tek bir varlık olarak gruplandırılmasıdır. Her varlık kendi "alanına" (domain) sahiptir ve ortak bir güvenlik özellikleri seti atanır.

- **Amaç:** Farklı güvenlik seviyelerindeki süreçlerin birbirine müdahale etmesini engellemektir.
- **Örnek (x86 İşlemci Ayrıcalık Seviyeleri):** İşlemci mimarisinde, işletim sistemi çekirdeği (kernel) en yetkili seviye olan **Ring 0**'da çalışır. Buna karşılık, kullanıcı modu uygulamaları en az yetkili seviye olan **Ring 3**'te çalışır. Bu sayede bir kullanıcı uygulaması, çekirdek seviyesindeki kritik işlemlere doğrudan erişemez.
- **İlişkili Model:** Alan ayrımı prensibi, daha önce gördüğümüz **Goguen-Meseguer Modeli**'nde de yer almaktadır.

2. Katmanlama (Layering)

Bir sistemin birçok soyut seviyeye veya katmana ayrılmasıdır. Bu yapı, güvenlik politikalarının farklı seviyelerde uygulanmasını ve operasyonların doğrulanmasını mümkün kılar.

- **Örnek 1 (OSI Modeli):** Ağ iletişimindeki 7 katmanlı OSI (Open Systems Interconnection) modelini düşünün. Her katman, bir üstündeki katmana belirli hizmetler sunar. Bu katmanlı yapı, güvenlik politikalarının uygulanmasını (örn: Ağ katmanında IP filtreleme, Uygulama katmanında içerik kontrolü) ve sistemin amaçlandığı gibi çalıştığının doğrulanmasını kolaylaştırır.
- **Örnek 2 (Programlama):** Bir programcı disk okuma/yazma işlemi yaparken genellikle kullandığı yüksek seviyeli programlama dilinin hazır fonksiyonlarını kullanır. Programlama dili, düşük seviyeli sistem çağrılarını gizler ve kullanıcı dostu yöntemler sunar.
- **İlişkili Kavram:** Katmanlama prensibi, **Derinlemesine Savunma (Defence-in-Depth)** kavramı ile doğrudan ilişkilidir.

3. Kapsülleme (Encapsulation)

Nesne Yönelimli Programlamada (OOP) sıkça kullanılan bu prensip, düşük seviyeli uygulamaları gizlemeyi ve bir nesnenin verisine doğrudan müdahaleyi önlemeyi amaçlar. Erişim, sadece bu amaç için yazılmış özel metotlar üzerinden sağlanır.

- **Örnek (Saat Nesnesi):** Bir "saat" nesneniz olduğunu varsayalım. Kullanıcının saniye değişkenine doğrudan erişip rastgele bir sayı yazmasına izin vermek yerine, bir `increment()` (artır) metodu sunarsınız.
- **Amaç:** Değişkenler için geçersiz değerlerin (örneğin saatin saniyesinin 75 olması gibi) girilmesini önlemektir.
- **Sistem Ölçeğinde Örnek:** Daha büyük sistemlerde, uygulamanızın veritabanına erişmesi için veritabanını doğrudan açmak yerine, uygun bir **API (Uygulama Programlama Arayüzü)** tasarlar veya kullanırsınız.

4. Yedeklilik (Redundancy)

Bu prensip, **Erişilebilirlik (Availability)** ve **Bütünlük (Integrity)** sağlamayı hedefler. Sistemin bir parçasının arızalanması durumunda diğer parçanın devreye girmesidir.

- **Örnek 1 (Donanım):** İki dahili güç kaynağına sahip bir sunucu düşünün. Güç kaynaklarından biri bozulursa, sistem diğer güç kaynağı üzerinden çalışmaya devam eder (Erişilebilirlik).
- **Örnek 2 (RAID 5):** Üç diskli bir RAID 5 yapılandırmasında, bir disk bozulsa bile kalan iki disk kullanılarak verilere ulaşılabilir. Ayrıca, disklerden birinde veri yanlış bir şekilde değiştirilirse, "parity" (eşlik biti) sayesinde bu bozulma tespit edilebilir ve verinin **bütünlüğü** korunur.

5. Sanallaştırma (Virtualization)

Bulut hizmetlerinin yaygınlaşmasıyla popüler hale gelen sanallaştırma, tek bir donanım setinin birden fazla işletim sistemi arasında paylaşılmasıdır.

- **Güvenlik Faydası:** Sanallaştırma, **Kum Havuzu (Sandboxing)** yetenekleri sağlar. Bu sayede güvenlik sınırları iyileştirilir. Örneğin, kötü amaçlı bir yazılımın güvenli bir şekilde patlatılması (detonation) ve davranışlarının gözlemlenmesi, ana sisteme zarar vermeden sanal ortamda yapılabilir.

B. Tasarım Prensipleri (Design Principles)

Bu kategori, sistemin işleyişi ve güvenliği sağlama yöntemleri üzerine odaklanan 5 prensibi içerir.

1. En Az Ayrıcalık (Least Privilege)

Bu prensip, gayri resmi olarak "ihtiyaç temeli" (need-to basis) veya "bilmesi gereken" (need-to-know basis) olarak da adlandırılır. "Kim, neye erişebilir?" sorusuna yanıt verir.

- **Kural:** Bir kullanıcının veya sistemin görevini yerine getirebilmesi için gereken **en az miktarda** izni vermelisiniz, fazlasını değil.
- **Örnek:** Bir kullanıcının sadece bir belgeyi görüntülemesi gerekiyorsa, ona "Okuma" hakkı verilmeli, ancak "Yazma" hakkı verilmemelidir.

2. Saldırı Yüzeyini En Aza İndirme (Attack Surface Minimisation)

Her sistem, saldırganların sistemi ele geçirmek için kullanabileceği güvenlik açıklarına sahiptir (bazıları bilinen, bazıları henüz keşfedilmemiş). Bu prensip, riskleri minimize etmeyi amaçlar.

- **Yöntem:** Gereksiz servisleri, portları veya özellikleri kapatmak.
- **Örnek (Linux Hardening):** Bir Linux sistemini sıkılaştırırken (hardening), ihtiyaç duyulmayan tüm servislerin devre dışı bırakılması standart bir adımdır.

3. Merkezi Parametre Doğrulama (Centralized Parameter Validation)

Pek çok tehdit, sistemin (özellikle kullanıcılardan) aldığı girdilerden kaynaklanır. Geçersiz girdiler, "Hizmet Reddi" (DoS) veya "Uzaktan Kod Yürütme" (RCE) gibi güvenlik açıklarını istismar etmek için kullanılabilir.

- **Çözüm:** Sistemin doğru durumda kalmasını sağlamak için parametre doğrulama şarttır. Bir sistemin işlediği çok sayıda parametre göz önüne alındığında, doğrulama işlemleri dağınık değil, **tek bir kütüphane veya sistem içinde merkezi olarak** yapılmalıdır.

4. Merkezi Genel Güvenlik Hizmetleri (Centralized General Security Services)

Güvenlik hizmetlerinin tek bir merkezden yönetilmesini hedefler.

- **Örnek:** Kimlik doğrulama (authentication) işlemleri için merkezi bir sunucu oluşturmak.
- **Uyarı:** Bu yöntemi uygularken, "Tek Hata Noktası" (Single Point of Failure) oluşturmamaya dikkat edilmeli ve erişilebilirliği sağlamak için uygun önlemler (yedeklilik vb.) alınmalıdır.

5. Hata ve İstisna İşleme İçin Hazırlık (Preparing for Error and Exception Handling)

Sistemler inşa edilirken hataların ve istisnaların kaçınılmaz olduğu kabul edilmelidir.

- **Senaryolar:**
 - Bir alışveriş uygulamasında müşterinin stokta olmayan bir ürünü sipariş etmeye çalışması.
 - Bir veritabanının aşırı yüklenip web uygulamasına yanıt vermeyi kesmesi.
- **"Fail Safe" (Güvenli Başarısızlık) İlkesi:** Sistemler hata verdiğinde güvenli bir durumda kalacak şekilde tasarlanmalıdır.
 - **Örnek:** Bir güvenlik duvarı (firewall) çökerse, trafiği serbest bırakmak yerine **tüm trafiği engellemelidir**.
- **Bilgi Sızıntısı Uyarısı:** Hata mesajları verilirken gizli bilgilerin sızdırılmamasına dikkat edilmelidir. Örneğin, bir hata sırasında bellek içeriğinin (dump) ekrana basılması, diğer müşterilere ait bilgilerin ifşa olmasına neden olabilir.

6. GÜVEN MODELLERİ: SIFIR GÜVEN VE "GÜVEN AMA DOĞRULA"

Güven, siber güvenlikte oldukça karmaşık bir konudur. Gerçek dünyada güven olmadan işleyişi sürdürmek neredeyse imkansızdır.

- **Örnek (Laptop Satıcısı):** Eğer dizüstü bilgisayar üreticinizin cihaza casus yazılım (spyware) yüklediğini düşünürseniz, muhtemelen tüm sistemi silip yeniden kurmanız gerekir.
- **Örnek (Donanım):** Eğer donanım üreticisine güvenmiyorsanız, o donanımı kullanmayı tamamen bırakırsınız.

İş dünyasında güven kavramı daha karmaşık hale gelse de, bize rehberlik edecek iki temel güvenlik prensibini inceleyeceğiz: **Güven ama Doğrula** ve **Sıfır Güven**.

1. Güven ama Doğrula (Trust but Verify)

Bu prensip, bir varlığa (kullanıcı veya sistem) ve davranışlarına güvensek bile, her zaman doğrulama yapmamız gerektiğini öğretir.

A. İşleyiş Mekanizması

Doğrulama işlemi genellikle düzgün **kayıt tutma (logging)** mekanizmalarının kurulmasını gerektirir. "Doğrulamak" demek, bu kayıtları (logları) inceleyerek her şeyin normal olduğundan emin olmak demektir.

B. Uygulamadaki Zorluklar ve Çözümler

Gerçek dünyada her şeyi manuel olarak doğrulamak mümkün değildir.

- **Senaryo:** Tek bir kullanıcının gün içinde gezindiği tüm internet sayfalarını tek tek incelemenin ne kadar büyük bir iş yükü olduğunu düşünün.
- **Çözüm:** Bu iş yükü nedeniyle, doğrulama süreci için **Proxy**, **Saldırı Tespit Sistemleri (IDS)** ve **Saldırı Önleme Sistemleri (IPS)** gibi otomatik güvenlik mekanizmaları kullanılır.

2. Sıfır Güven (Zero Trust)

Sıfır Güven prensibi, "güven" kavramını bir **zafiyet (vulnerability)** olarak görür ve bu nedenle onu ortadan kaldırmaya çalışır. Özellikle **iç tehditlere (insider threats)** karşı etkilidir.

A. Temel Felsefe

Bu yaklaşım dolaylı olarak şu öğretiyi sunar: **"Asla güvenme, her zaman doğrula."** (Never trust, always verify).

- Her varlık, aksi kanıtlanana kadar **düşman (adversarial)** olarak kabul edilir.
- Konuma veya sahipliğe dayalı güven yoktur. Sıfır Güven, bir cihaza sadece "şirket iç ağında" olduğu veya "şirket malı" olduğu için güvenmez. (Bu durum, iç ağlara otomatik olarak güvenen eski modellerle tezat oluşturur).

B. Gereklilikler ve Faydalar

Herhangi bir kaynağa erişim sağlanmadan önce mutlaka **Kimlik Doğrulama (Authentication)** ve **Yetkilendirme (Authorization)** gereklidir.

- **Neden Önemli? (Neden-Sonuç):** Eğer bir veri ihlali gerçekleşirse, Sıfır Güven mimarisi sayesinde hasar çok daha sınırlı kalır (contained). Çünkü saldırgan sisteme girse bile içeride rahatça hareket edemez.

C. Mikrosegmentasyon (Microsegmentation)

Sıfır Güven'in uygulanmasındaki en önemli tekniklerden biridir.

- **Tanım:** Ağ segmentlerinin tek bir ana bilgisayara (host) kadar küçültülebileceği bir ağ tasarımıdır.
- **İşleyiş:** Segmentler arasındaki her iletişim; kimlik doğrulama, erişim kontrol listesi (ACL) kontrolleri ve diğer güvenlik gereksinimlerine tabidir.

3. Uygulama Dengesi

Sıfır Güven modelini iş süreçlerini olumsuz etkilemeden (iş durdurmadan) uygulamanın bir sınırı vardır. Ancak bu sınırın varlığı, modelin uygulanabilir olduğu her noktada uygulanmaması gerektiği anlamına gelmez. Mümkün olan her yerde Sıfır Güven prensipleri devreye alınmalıdır.

7. TEHDİT VE RİSK KAVRAMLARI (THREAT VS. RISK)

Kafa karışıklığını önlemek için aşağıdaki üç terimin arasındaki farkı net bir şekilde anlamamız gerekir.

1. Temel Tanımlar

- Zafiyet (Vulnerability):** Saldırıya veya hasara karşı duyarlı olma durumudur. Bilgi güvenliğinde zafiyet, sistemdeki bir **zayıflığı** (weakness) ifade eder.
- Tehdit (Threat):** Bu zayıflık veya zafiyetle ilişkili **potansiyel tehlikedir**.
- Risk (Risk):** Bir tehdit aktörünün bir zafiyeti istismar etme **olasılığı** (likelihood) ve bunun iş üzerindeki **etkisinin** (impact) bileşimidir.

2. Senaryo Analizleri

Bu kavramların nasıl birbirine bağlı olduğunu anlamak için biri fiziksel, diğeri dijital dünyadan iki örneği inceleyelim.

Senaryo A: Camlı Mağaza (Fiziksel Güvenlik Örneği)

Kapıları ve pencereleri standart camdan yapılmış bir galeri veya mağazayı (showroom) düşünelim.

- Zafiyet (Vulnerability):** Camın doğası gereği kırılabilir olması, sistemin **zayıflığıdır**.
- Tehdit (Threat):** Cam kapı ve pencerelerin kırılabilecek olması, bu zayıflıktan doğan **potansiyel tehlikedir**.
- Risk Analizi:** Mağaza sahipleri riski hesaplarken şunları düşünmelidir:
 - Bir camın kırılma **olasılığı** nedir? (Örn: Bölgenin suç oranı, hava durumu vb.)
 - Kırılırsa iş üzerindeki **etkisi** ne olur? (Örn: Çalınan mallar, tamir masrafı, iş durması).

Senaryo B: Hastane Veritabanı (Bilişim Sistemleri Örneği)

Tüm tıbbi kayıtları saklamak için belirli bir veritabanı sistemini kullanan bir hastanede çalıştığınızı varsayalım.

- Olay:** Bir gün güvenlik haberlerini takip ederken, hastanede kullandığınız veritabanı sisteminin sadece **savunmasız (zafiyet barındıran)** olmadığını, aynı zamanda bu açığı kullanan bir kavram kanıtı (proof-of-concept) saldırı kodunun da yayınlandığını öğreniyorsunuz.

2. **Tehdit (Threat):** Saldırı kodunun (exploit) yayınlanmış olması, tehdidin artık teorik değil, **gerçek** ve yakın olduğunu gösterir.
3. **Risk Kararı:** Bu bilgi ışığında, hastane yönetimi riski (saldırının gerçekleşme olasılığı ve hasta verilerinin çalınmasının yaratacağı yıkıcı etkiyi) değerlendirmeli ve bir sonraki adımlara (yamama, sistemi kapatma vb.) karar vermelidir.